

PROJECT: AltRix School ERP SaaS

We are NOT planning architecture anymore.

We are now moving into IMPLEMENTATION MODE.

Your task is to FULLY migrate the backend layer of the EXISTING AltRix system from direct Supabase business logic into a COMPLETE PRODUCTION-READY FastAPI backend.

IMPORTANT:

Do NOT redesign the product.

Do NOT create mock implementations.

Do NOT create placeholders.

Do NOT create architecture-only documentation.

Do NOT stop after planning.

Build the ACTUAL WORKING BACKEND.

Storage migration is NOT required for now.

Everything else must be production-ready.

---

## CURRENT STATE

Frontend:

- Lovable React
- TypeScript
- Tailwind
- Existing UI already built

Database:

- Existing Supabase PostgreSQL
- Existing schema
- Existing data

Goal:

Frontend ↓

FastAPI

↓

Supabase PostgreSQL

Supabase becomes DATABASE ONLY.

Business logic must move into FastAPI.

---

## PHASE 1 — PROJECT DISCOVERY

First analyze:

- complete schema
- ERD
- migrations
- tables
- columns
- relationships
- RLS
- RPCs
- edge functions
- auth logic
- role logic
- permissions
- owner system
- campus system
- parent-child system

Generate migration map internally.

Then start implementation.

Do not stop after analysis.

---

## PHASE 2 — FASTAPI FOUNDATION

Create:

- FastAPI application
- modular structure

- environment configuration
- dependency injection
- database session handling
- logging
- configuration management

Implement:

- production configuration
  - development configuration
- 

### PHASE 3 — DATABASE LAYER

Generate:

- SQLAlchemy models
- all relationships
- indexes
- constraints

for ALL tables.

No table skipped.

No module skipped.

---

### PHASE 4 — AUTHENTICATION SYSTEM

Implement:

- JWT Authentication
- refresh tokens
- secure login
- secure logout
- session validation
- password reset

Preserve existing user accounts.

Preserve existing role assignments.

---

### PHASE 5 — PERMISSION ENGINE

Implement COMPLETE permission architecture.

Roles become permission bundles.

Support:

- module permissions
- action permissions
- route permissions

Dynamic resolution required.

Must support:

- multiple roles
  - campus permissions
  - parent-child permissions
  - owner permissions
- 

## PHASE 6 — MULTI ROLE SYSTEM

Support:

- Super Admin
- School Owner
- Principal
- Vice Principal
- School Admin
- Teacher
- Parent
- Student
- HR
- Accountant
- Marketing
- Counselor

Multiple roles on one account must work.

Permission merging required.

---

## PHASE 7 — MULTI CAMPUS SYSTEM

Support:

- multiple schools
- multiple campuses
- campus switchers

- owner-level access

Data isolation required.

---

## PHASE 8 — CORE MODULE APIs

Generate complete APIs for:

Admissions

Students

Parents

Teachers

HR

Attendance

Exams

Results

Diary

Notices

Holidays

Messaging

Notifications

Finance

Fees

Invoices

Payments

Reports

AI Timetable

Behavior Tracking

Owner Management

Campus Management

Settings

Branding

Audit Logs

Every CRUD operation must work.

---

## PHASE 9 — REPORTING SYSTEM

Implement APIs for:

- report cards
- PDF exports
- fee reports
- attendance reports
- finance reports
- campus reports

Production-ready.

---

## PHASE 10 — REALTIME FEATURES

Implement:

- WebSockets
- notifications
- messaging
- alerts

using FastAPI-compatible realtime stack.

---

## PHASE 11 — AI LAYER

Implement backend services for:

- AI timetable
- AI insights
- AI analytics

Create service architecture.

---

## PHASE 12 — PAYMENT SYSTEM

Implement production-ready:

- JazzCash integration layer
- payment verification
- payment callbacks
- fee synchronization

No placeholders.

---

## PHASE 13 — AUDIT SYSTEM

Implement:

- activity logs
- security logs
- change tracking

for all critical modules.

---

## PHASE 14 — FRONTEND INTEGRATION

Replace direct Supabase business logic calls.

Frontend should call FastAPI.

Preserve existing UI.

Preserve existing workflows.

Do not break screens.

---

## PHASE 15 — ROUTE SECURITY

Protect all APIs.

Validate:

- permissions
- campuses
- ownership
- parent-child relationships

Block unauthorized access.

---

## PHASE 16 — OPTIMIZATION

Optimize:

- queries
- joins
- permissions
- API performance

Avoid:

- N+1 queries
  - duplicate fetches
- 

## PHASE 17 — TESTING

Test:

Single-role users

Multi-role users

Parents with multiple children

Owners with multiple campuses

Finance workflows

Admissions workflows

Messaging workflows

Attendance workflows



Result workflows

Reports workflows

---

## PHASE 18 — PRODUCTION READINESS

Deliver:

- working FastAPI backend
- working database integration
- working authentication
- working permissions
- working APIs
- working realtime features
- working finance system
- working reporting system

No placeholders.

No stubs.

No TODOs.

No pseudo code.

No unfinished modules.

---

## FINAL RESULT REQUIRED

A complete production-ready FastAPI backend running against the existing Supabase PostgreSQL database.

The frontend must continue working.

The database must remain intact.

All existing modules must function through FastAPI.

No architecture-only output.

No planning-only output.

Implement the actual backend completely and deliver working code.